

Fiche 318 — Système d'exploitation : fichiers, chemins, commandes et archives

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 14 « OS facilities » (§14.1 fichiers et répertoires, §14.2 chemins, §14.3 commandes système, §14.4 compression et archives)
Difficulté	Intermédiaire — peu de théorie, beaucoup de fonctions et de pièges de portabilité
Temps d'étude estimé	45 min
Prérequis	Fiches 300 et 307 (session, répertoire de travail, lecture de fichiers)
Concepts clés	R comme langage de script , <code>file.create</code> / <code>dir.create</code> , <code>tempfile</code> , <code>file.remove</code> / <code>unlink</code> , <code>list.files</code> / <code>dir</code> / <code>list.dirs</code> , <code>Sys.glob</code> , <code>file.info</code> , <code>file.exists</code> / <code>file.access</code> / <code>file_test</code> , <code>file.copy</code> , <code>file.choose</code> , <code>file.show</code> / <code>file.edit</code> , <code>basename</code> / <code>dirname</code> / <code>file.path</code> , <code>path.expand</code> , <code>normalizePath</code> , <code>shortPathName</code> , <code>Sys.chmod</code> , <code>system</code> / <code>system2</code> , <code>Sys.which</code> , <code>shQuote</code> , connexions et détection de compression, <code>gzip</code> / <code>bzip2</code> / <code>xz</code> , <code>untar</code> / <code>unzip</code>
À retenir en priorité	<code>file.path()</code> pour assembler un chemin · <code>system2</code> pour du code portable · <code>shQuote</code> avant toute commande · la sensibilité à la casse dépend du système · <code>file</code> détecte la compression toute seule.

Vue d'ensemble

POURQUOI	« R has quite extensive facilities to access the OS ... this allows it to be USED AS A SCRIPTING LANGUAGE and that ability is MUCH USED BY R ITSELF, for example TO INSTALL PACKAGES. » « considerable effort has gone into make the scripting facilities AS PLATFORM-INDEPENDENT AS IS FEASIBLE »
CREER	<code>file.create</code> <code>dir.create</code> (= touch et mkdir) <code>tempfile</code> pour le repertoire de session
SUPPRIMER	<code>file.remove</code> <code>unlink</code> (unlink enleve des ARBORESCENCES)
LISTER	<code>list.files</code> / <code>dir</code> <code>list.dirs</code> -> par EXPRESSION REGULIERE

```

Sys.glob                                -> par JOKERS
INFORMER    file.info      file.exists / file.access / file_test
COPIER      file.copy      (= cp)
CHOISIR     file.choose    choose.files / choose.dir (Windows)
            tk_choose.files / tk_choose.dir (paquet tcltk)
AFFICHER    file.show      file.edit
LIENS       file.link      Sys.readlink

CHEMINS     basename  dirname -> file.path POUR ASSEMBLER
            path.expand (le tilde)      normalizePath (chemin canonique)
            shortPathName (Windows, noms « 8.3 », sans espaces)
            Sys.chmod pour les permissions

PIEGES DE PORTABILITE

casse      POSIX sensible ; Windows et macOS INSENSIBLES par default
separateur Windows accepte / et \ ; R convertit les exceptions
slash final NON VALIDE sous Windows -- a EVITER
slashes multiples /abc//def valide en POSIX ; double slash INITIAL special
UNC        \\server\... NON SUPPORTE
d:foo/bar  relatif au repertoire courant DU LECTEUR -- eviter

COMMANDES  system  system2  -- « system2 ... EASIER TO WRITE
            CROSS-PLATFORM CODE »
            Sys.which (la commande existe-t-elle ?)  shQuote (quoter)

COMPRESSION file() identifie la compression par L'EN-TETE « MAGIC »
            gzip (bon compromis) · compress (ancien) · bzip2 · xz
            untar  unzip

```

Le problème posé. « *R has quite extensive facilities to access the OS under which it is running : this allows it to be used as a scripting language and that ability is much used by R itself, for example to install packages.* » (§14)

INTUITION — POURQUOI CE CHAPITRE EXISTE.

R n'est pas seulement un langage d'analyse : **il s'administre lui-même en R**. `install.packages()` (fiche 315), `R CMD build` (fiche 300) — ces outils **sont écrits en R** et manipulent des fichiers, lancent des commandes, dépaquettent des archives. Le chapitre 14 est **l'outillage qui rend cela possible**.

« *Because R's own scripts need to work across all platforms, considerable effort has gone into make the scripting facilities as platform-independent as is feasible.* » — le mot « **as is feasible** » est important : le §14.2 énumère tout ce qui **n'a pas pu** être uniformisé.

● Concept 1 — Fichiers et répertoires

Cadrage (§14.1). « *There are **many functions to manipulate files and directories**. Here are pointers to some of the more commonly used ones.* » — le chapitre est **un index commenté**, pas une documentation exhaustive.

Besoin	Fonctions (§14.1)	Équivalent cité
Créer	<code>file.create</code> , <code>dir.create</code>	« the analogues of the POSIX utilities <code>touch</code> and <code>mkdir</code> »
Fichiers temporaires	<code>tempfile</code>	« for temporary files and directories in the R session directory »
Supprimer	<code>file.remove</code> , <code>unlink</code>	« the latter can remove directory trees »
Lister	<code>list.files</code> (aussi <code>dir</code>), <code>list.dirs</code>	« These can select files using a regular expression »
Lister par jokers	<code>Sys.glob</code>	« to select by wildcards »
S'informer	<code>file.info</code>	« Many types of information on a filepath (including for example if it is a file or directory) »
Tester l'existence	<code>file.exists</code> , <code>file.access</code> , <code>file_test</code>	« <code>file_test</code> is a version of the POSIX <code>test</code> command for those familiar with shell scripting »
Copier	<code>file.copy</code>	« the R analogue of the POSIX command <code>cp</code> »
Choisir	<code>file.choose</code>	Windows : <code>choose.files</code> , <code>choose.dir</code> ; paquet <code>tcltk</code> : <code>tk_choose.files</code> , <code>tk_choose.dir</code>
Afficher / éditer	<code>file.show</code> , <code>file.edit</code>	« in a way appropriate to the R port, using the facilities of a console (such as RGui on Windows or R.app on macOS) »
Liens	<code>file.link</code> , <code>Sys.readlink</code>	« There is some support for links in the filesystem »

Deux distinctions à ne pas manquer.

- `file.remove` contre `unlink` : le second « *can remove directory trees* ». C'est **plus puissant et plus dangereux** — l'équivalent d'un `rm -r`.
- Trois fonctions pour « le fichier existe-t-il ? » : « *There are several ways to find out if a file "exists" (a file can exist on the filesystem and not be visible to the current user). There are functions `file.exists` , `file.access` and `file_test` with various versions of this test.* »

La parenthèse est le cœur du problème : « exister » et « être accessible » sont **deux questions différentes**. `file.exists` répond à la première, `file.access` à la seconde.

Rappel — le lien avec la fiche 307. `list.files()` sélectionne par expression régulière, `sys.glob()` par **jokers** (`*.csv`). Deux langages différents pour deux habitudes différentes : celle de R, celle du shell.

● Concept 2 — Les chemins et leurs pièges

Cadrage (§14.2). « *With a few exceptions, R relies on the underlying OS functions to manipulate filepaths. Some aspects of this are allowed to depend on the OS, and do, even down to the version of the OS. There are POSIX standards for how OSes should interpret filepaths and many R users assume POSIX compliance : but Windows does not claim to be compliant and other OSes may be less than completely compliant.* »

2.1 Les six pièges énumérés par le cours

Règle (§14.2). « *The following are some issues which have been encountered with filepaths.* »

#	Piège	Ce que dit le cours
1	La casse	« POSIX filesystems are case-sensitive , so <code>foo.png</code> and <code>Foo.PNG</code> are different files. However, the defaults on Windows and macOS are to be case-insensitive, and FAT filesystems (commonly used on removable storage) are not normally case-sensitive (and all filepaths may be mapped to lower case). »
2	Le séparateur	« Almost all the Windows' OS services support the use of slash or backslash as the filepath separator, and R converts the known exceptions to the form required by Windows. »
3	Le slash final	« The behaviour of filepaths with a trailing slash is OS-dependent. Such paths are not valid on Windows and should not be expected to work. POSIX-2008 requires such paths to match only directories, but earlier versions allowed them to also match files. So they are best avoided. »
4	Les slashes multiples	« Multiple slashes in filepaths such as <code>/abc//def</code> are valid on POSIX filesystems and treated as if there was only one slash. They are usually accepted by Windows' OS functions. However, leading double slashes may have a different meaning. »
5	Les chemins UNC	« Windows' UNC filepaths (such as <code>\\server\dir1\dir2\file</code>) are not supported, but they may work in some R functions. POSIX filesystems are allowed to treat a leading double slash specially. »
6	Les chemins relatifs à un lecteur	« Windows allows filepaths containing drives and relative to the current directory on a drive, e.g. <code>d:foo/bar</code> refers to <code>d:/a/b/c/foo/bar</code> if the current directory on drive <code>d:</code> is <code>/a/b/c</code>. It is intended that these work, but the use of absolute paths is safer. »

EN CLAIR — CE QUE CES SIX POINTS ONT EN COMMUN.

Chacun décrit un endroit où **l'intuition du programmeur ne correspond pas à ce que fait le système**. Le premier surprend le plus : un script développé sur macOS, où `Data.csv` et `data.csv` désignent le même fichier, **échoue sous Linux** où ce sont deux fichiers.

La règle de survie tient en une ligne : ne construire des chemins **qu'avec les fonctions dédiées**, et préférer **les chemins absolus**.

2.2 Les fonctions de chemin

Règle (§14.2). « **Functions `basename` and `dirname` select parts of a file path : the recommended way to assemble a file path from components is `file.path`.** »

```
file.path("data", "brut", "mesures.csv") # le séparateur est choisi par R
```

C'est la fonction la plus importante du chapitre. Concaténer à la main — `paste0(dossier, "/", nom)` — c'est **coder en dur un séparateur** et se heurter au piège 2. `file.path()` **fait le bon choix pour le système courant**.

Les autres (§14.2).

- `path.expand` — « *does "tilde expansion", substituting values for home directories (the current user's, and perhaps those of other users)* ».
- `normalizePath` — « *On filesystems with links, a single file can be referred to by many filepaths. `normalizePath` will find a canonical filepath.* »
- `shortPathName` — « *Windows has the concepts of short ("8.3") and long file names : `normalizePath` will return an absolute path using long file names and `shortPathName` will return a version using short names. The latter does not contain spaces and uses backslash as the separator, so is sometimes useful for exporting names from R.* »

MÉTHODE — LE CAS DES ESPACES.

Un chemin contenant des espaces casse la plupart des commandes système. Deux réponses selon le contexte : `shQuote` (concept 3) pour passer par un shell, et `shortPathName` sous Windows — « *does not contain spaces* » — pour transmettre un nom à un programme extérieur.

Les permissions (§14.2). « *File permissions are a related topic. R has support for the POSIX concepts of read/write/execute permission for owner/group/all but this may be only partially supported on the filesystem — so for example on Windows only read-only files (for the account running the R session) are recognized. Access Control Lists (ACLs) are employed on several filesystems, but do not have an agreed standard and R has no facilities to control them. Use `Sys.chmod` to change permissions.* »

● Concept 3 — Lancer une commande système

DÉFINITION (§14.3).

« Functions `system` and `system2` are used to invoke a system command and optionally collect its output. `system2` is a little more general but its main advantage is that it is easier to write cross-platform code using it. »

POURQUOI SYSTEM DIFFÈRE SELON LE SYSTÈME (§14.3).

« `system` behaves differently on Windows from other OSes (because the API C call of that name does). Elsewhere it invokes a shell to run the command : the Windows port of R has a function `shell` to do that. »

Ailleurs qu'à Windows, `system` passe par un shell — donc l'expansion des jokers, les redirections, les pipes fonctionnent. **Sous Windows, non** : il faut `shell()`. `system2` sépare l'exécutable de ses arguments et **contourne la question**.

Besoin	Fonction
Lancer une commande, code portable	<code>system2</code>
Lancer via un shell	<code>system</code> (hors Windows) · <code>shell</code> (Windows)
La commande existe-t-elle ?	<code>Sys.which</code>
Protéger un chemin ou un argument	<code>shQuote</code>

Règle (§14.3). « To find out if the OS includes a command, use `Sys.which`, which attempts to do this in a cross-platform way (unfortunately it is not a standard OS service). »

« Function `shQuote` will quote filepaths as needed for commands in the current OS. »

`shQuote` est le pendant de `file.path` : l'un assemble un chemin correctement, l'autre le **protège** correctement avant de le confier à un shell. Les deux évitent d'écrire soi-même des règles qui diffèrent d'un système à l'autre.

● Concept 4 — Compression et archives

DÉFINITION (§14.4).

« *Recent versions of R have extensive facilities to read and write compressed files, often transparently. Reading of files in R is to a very large extent done by connections, and the `file` function which is used to open a connection to a file (or a URL) and is able to identify the compression used from the "magic" header of the file.* »

C'est la phrase à retenir : on n'a **rien à déclarer**. Ouvrir un fichier compressé se fait comme ouvrir un fichier ordinaire — `file()` **lit l'en-tête et décide**.

Les quatre formats (§14.4).

Format	Ce que dit le cours
gzip	« <i>The type of compression which has been supported for longest ... and that remains a good general compromise</i> »
compress	« <i>Files compressed by the earlier Unix <code>compress</code> utility can also be read, but these are becoming rare</i> »
bzip2 et xz	« <i>These generally achieve higher rates of compression (depending on the file, much higher) at the expense of slower decompression and much slower compression</i> »

« *There is some confusion between `xz` and `Lzma` compression ... R can read files compressed by most versions of either.* »

Le compromis, énoncé par le cours. `gzip` = « *a good general compromise* » ; `bzip2/xz` = **meilleure compression**, mais **décompression plus lente** et **compression bien plus lente**. Le choix dépend donc de ce qu'on fait le plus souvent : **écrire une fois et lire souvent** favorise `xz` ; **écrire souvent** favorise `gzip`.

Les archives (§14.4). « *File archives are single files which contain a collection of files, the most common ones being "tarballs" and zip files as used to distribute R packages. R can list and unpack both (see functions `untar` and `unzip`) and create both (for zip with the help of an external program).* »

L'asymétrie de la dernière phrase : R **lit et écrit** les deux, mais **la création d'un zip demande un programme externe**. C'est cohérent avec le §14.3 — d'où l'intérêt de `sys.which` pour vérifier sa présence.

Énoncé. Écrire un script qui, à partir d'un dossier, dépaquette une archive, liste les fichiers `.csv` compressés qu'elle contient, les lit, et écrit un fichier de synthèse — le tout devant fonctionner sous Windows comme sous Linux.

Étape 1 — *ne jamais concaténer un chemin à la main.* « **the recommended way to assemble a file path from components is `file.path`** » (§14.2).

```
racine <- normalizePath(".")
archive <- file.path(racine, "donnees.tar.gz")
```

`normalizePath` parce que « **on filesystems with links, a single file can be referred to by many filepaths** » — on veut **un chemin canonique**, pas une des formes possibles.

Étape 2 — *préparer un espace de travail temporaire.* « For temporary files and directories **in the R session directory** see `tempfile` » (§14.1) :

```
travail <- tempfile()
dir.create(travail)
on.exit(unlink(travail, recursive = TRUE))
```

Deux points. `unlink` est choisi parce que « **the latter can remove directory trees** » — `file.remove` ne suffirait pas. Et `on.exit` (fiche 317) garantit le nettoyage « **either directly or as the result of a warning** ».

Étape 3 — *dépaqueter.* « **R can list and unpack both (see functions `untar` and `unzip`)** » (§14.4) :

```
untar(archive, exdir = travail)
```

Étape 4 — *lister*. Deux langages au choix (§14.1) : **expression régulière** avec `list.files` , **jokers** avec `Sys.glob` .

```
fichiers <- list.files(travail, pattern = "\\\\.csv\\.gz$", full.names = TRUE)
# ou : Sys.glob(file.path(travail, "*.csv.gz"))
```

Le point est échappé dans l'expression régulière : `\\.` — sinon il apparait n'importe quel caractère.

Étape 5 — *lire, sans se soucier de la compression*. C'est le point le plus reposant du chapitre :

```
donnees <- lapply(fichiers, read.csv)
```

« *file ... is able to identify the compression used from the "magic" header of the file* » — **aucun argument à passer**. `read.csv` (fiche 307) ouvre une connexion, et la connexion décide.

Étape 6 — *vérifier avant d'agir*. Si une étape doit appeler un programme extérieur — créer un zip, par exemple, ce que R ne fait « *with the help of an external program* » —, vérifier d'abord :

```
if (nzchar(Sys.which("zip"))) { ... }
```

« *To find out if the OS includes a command, use `Sys.which` , which attempts to do this in a cross-platform way.* »

Étape 7 — *protéger les arguments*. Si l'on passe par `system2` , tout chemin susceptible de contenir un espace doit être **protégé** :

```
system2("zip", c("-r", shQuote(sortie), shQuote(travail)))
```

« **Function `shQuote` will quote filepaths as needed for commands in the current OS.** » Et `system2` plutôt que `system` , parce qu'« **its main advantage is that it is easier to write cross-platform code using it** ».

Étape 8 — *la casse*. Développé sous Windows ou macOS, ce script marchera avec `Donnees.CSV` ;
sous Linux, non — « *POSIX filesystems are case-sensitive* ». Le remède : **ne jamais compter sur l'insensibilité**, et si l'entrée est incertaine, lister puis filtrer avec `ignore.case = TRUE` .

Étape 9 — *les chemins absolus*. Le cours conclut son point 6 par « *the use of absolute paths is safer* ».
Un script qui dépend du répertoire courant dépend de **la manière dont il a été lancé** (fiche 300)
— c'est une fragilité gratuite.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« créer un dossier »	<code>dir.create</code>
« un fichier temporaire »	<code>tempfile</code>
« effacer une arborescence »	<code>unlink</code> , pas <code>file.remove</code>
« tous les fichiers <code>.csv</code> »	<code>list.files(pattern=)</code> ou <code>Sys.glob</code>
« est-ce un dossier ? »	<code>file.info</code>
« le fichier existe-t-il ? »	<code>file.exists</code> — et <code>file.access</code> pour l'accès
« comme la commande <code>test</code> du shell »	<code>file_test</code>
« copier un fichier »	<code>file.copy</code>
« demander un fichier à l'utilisateur »	<code>file.choose</code> (ou <code>choose.files</code> sous Windows)
« assembler un chemin »	<code>file.path</code> , jamais <code>paste0</code>
« le nom seul, le dossier seul »	<code>basename</code> , <code>dirname</code>
« le tilde n'est pas résolu »	<code>path.expand</code>
« plusieurs chemins pour le même fichier »	<code>normalizePath</code>
« le chemin contient des espaces »	<code>shQuote</code> , ou <code>shortPathName</code> sous Windows
« changer les permissions »	<code>Sys.chmod</code>
« ça marche chez moi, pas sous Linux »	la casse
« lancer une commande système »	<code>system2</code> pour du code portable
« avec des jokers ou des redirections »	un shell : <code>system</code> (hors Windows), <code>shell</code> (Windows)
« la commande est-elle installée ? »	<code>Sys.which</code>
« lire un fichier compressé »	rien à faire — la connexion le détecte
« quel format de compression choisir ? »	gzip par défaut ; xz/bzip2 si l'on écrit rarement

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« dépaqueter une archive »	<code>untar</code> , <code>unzip</code>
« créer un zip »	un programme externe est nécessaire

Comment résoudre ce type d'exercice

Protocole « écrire un script de fichiers portable » — 5 étapes.

1. **Assembler tous les chemins** par `file.path()` — jamais de séparateur écrit à la main.
2. **Préférer les chemins absolus**, obtenus par `normalizePath()` .
3. **Ne jamais compter sur l'insensibilité à la casse** ni sur un **slash final**.
4. Pour le travail intermédiaire, `tempfile()` + `dir.create()` , nettoyé par `on.exit(unlink(..., recursive = TRUE))` .
5. Vérifier l'existence **et** l'accès : `file.exists` **puis** `file.access` .

Protocole « appeler un programme extérieur » — 4 étapes.

1. `Sys.which("prog")` d'abord — la commande peut ne pas exister.
2. `system2` plutôt que `system` : « *easier to write cross-platform code* ».
3. `shQuote()` sur tout argument susceptible de contenir un espace.
4. Si l'on a besoin d'un shell (jokers, tubes) : `system` hors Windows, `shell` sous Windows.

● Common mistakes

Erreur	Correction
Construire un chemin par <code>paste0(dir, "/", f)</code>	<code>file.path()</code> choisit le bon séparateur
Utiliser <code>file.remove</code> sur un dossier plein	<code>unlink</code> retire les arborescences
Confondre « existe » et « accessible »	« <i>a file can exist ... and not be visible to the current user</i> »
Confondre expression régulière et jokers	<code>list.files(pattern=)</code> contre <code>Sys.glob</code>
Oublier d'échapper le point dans un motif	<code>"\\.csv\$"</code> , sinon il apparie tout caractère
Compter sur l'insensibilité à la casse	POSIX est sensible
Terminer un chemin par un slash	« <i>not valid on Windows ... best avoided</i> »
Utiliser un chemin UNC	« <i>not supported</i> »
Utiliser <code>d:foo/bar</code>	« <i>the use of absolute paths is safer</i> »
Croire un double slash initial anodin	« <i>leading double slashes may have a different meaning</i> »
Oublier <code>path.expand</code> sur un <code>~</code>	le tilde n'est pas toujours résolu
Passer un chemin à espaces sans protection	<code>shQuote</code> — ou <code>shortPathName</code> sous Windows
Utiliser <code>system</code> en croyant écrire du portable	<code>system2</code> est fait pour cela
Attendre un shell de <code>system</code> sous Windows	il faut <code>shell</code>
Appeler un programme sans vérifier sa présence	<code>Sys.which</code>
Décompresser à la main avant de lire	la connexion détecte l'en-tête magique
Choisir <code>xz</code> pour des écritures fréquentes	« <i>much slower compression</i> »
Croire R capable de créer un zip seul	« <i>with the help of an external program</i> »
Croire R capable de gérer les ACL	« <i>R has no facilities to control them</i> »

Ultimate Review

Pourquoi ce chapitre. « *R has quite extensive facilities to access the OS ... this allows it to be used as a scripting language and that ability is much used by R itself, for example to install packages.* » Et : « *considerable effort has gone into make the scripting facilities as platform-independent as is feasible* ».

Créer. `file.create`, `dir.create` — « *the analogues of the POSIX utilities `touch` and `mkdir`* » · `tempfile` pour le répertoire de session.

Supprimer. `file.remove` et `unlink` — « *the latter can remove directory trees* ».

Lister. `list.files` (= `dir`) et `list.dirs` — « *can select files using a regular expression* » · `Sys.glob` — « *to select by wildcards* ».

S'informer. `file.info` — « *many types of information ... including for example if it is a file or directory* » · `file.exists`, `file.access`, `file_test` (« *a version of the POSIX `test` command* ») — « *a file can exist on the filesystem and not be visible to the current user* ».

Autres. `file.copy` (= `cp`) · `file.choose`, et sous Windows `choose.files` / `choose.dir`, dans `tcltk` `tk_choose.files` / `tk_choose.dir` · `file.show` et `file.edit`, « *in a way appropriate to the R port* » · `file.link`, `Sys.readlink`.

Les six pièges de chemin (§14.2). (1) **la casse** — POSIX **sensible**, Windows et macOS **insensibles par défaut**, FAT **insensible et parfois tout en minuscules** ; (2) **le séparateur** — Windows accepte **les deux**, « *R converts the known exceptions* » ; (3) **le slash final** — « *not valid on Windows ... best avoided* » ; (4) **les slashes multiples** — valides en POSIX, mais « *leading double slashes may have a different meaning* » ; (5) **UNC** — « *not supported* » ; (6) `d:foo/bar` — « *the use of absolute paths is safer* ».

Fonctions de chemin. `basename`, `dirname` — et « *the recommended way to assemble a file path from components is `file.path`* » · `path.expand` pour le tilde · `normalizePath` pour un **chemin canonique** · `shortPathName` (Windows, noms 8.3) — « *does not contain spaces and uses backslash as the separator, so is sometimes useful for exporting names from R* ».

Permissions. POSIX lecture/écriture/exécution pour propriétaire/groupe/tous, « *may be only partially supported* » — sous Windows, **seul le lecture seule** est reconnu · **ACL** : « *do not have an agreed standard and R has no facilities to control them* » · `Sys.chmod`.

Commandes système. `system` et `system2` — « *system2 is a little more general but its main advantage is that it is easier to write cross-platform code using it* » · « *system behaves differently on Windows ... Elsewhere it invokes a shell ... the Windows port has `shell`* » · `Sys.which` — « *attempts to do this in a cross-platform way (unfortunately it is not a standard OS service)* » · `shQuote` — « *will quote filepaths as needed for commands in the current OS* ».

Compression. « *Reading of files in R is to a very large extent done by connections, and the `file` function ... is able to identify the compression used from the "magic" header of the file* » — donc transparente · **gzip** « *remains a good general compromise* » · **compress** — « *becoming rare* » · **bzip2** et **xz** — « *higher rates of compression ... at the expense of slower decompression and much slower compression* » · « *some confusion between `xz` and `Lzma` ... R can read files compressed by most versions of either* ».

Archives. « *the most common ones being "tarballs" and zip files as used to distribute R packages* » · **untar** et **unzip** pour lister et dépaqueter · créer un zip demande **un programme externe**.

Active Recall

« *R has quite extensive facilities to access the OS under which it is running : this allows it to be used as a scripting language and that ability is much used by R itself, for example to install packages.* » (§14)

R s'administre en R. `install.packages()` (fiche 315) et les outils `R CMD` (fiche 300) manipulent des fichiers, lancent des commandes, dépaquettent des archives — **en R**.

Et c'est ce qui explique l'effort de portabilité : « *Because R's own scripts need to work across all platforms, considerable effort has gone into make the scripting facilities as platform-independent as is feasible.* »

« **As is feasible** » — le §14.2 énumère précisément ce qui **n'a pas pu** être uniformisé.

« *Files can be removed by either `file.remove` or `unlink` : the latter can remove directory trees.* » (§14.1)

unlink est l'équivalent d'un `rm -r` : il descend dans les sous-dossiers. `file.remove` ne traite que des fichiers.

Conséquence pratique : pour nettoyer un répertoire temporaire créé par `tempfile()` + `dir.create()`, c'est `unlink(dir, recursive = TRUE)` qu'il faut — de préférence dans un `on.exit()` (fiche 317) pour que le nettoyage ait lieu même en cas d'erreur.

Et la contrepartie : c'est aussi la fonction avec laquelle on efface le plus de choses par accident.

« *There are several ways to find out if a file "exists" (a file can exist on the filesystem and not be visible to the current user). There are functions `file.exists` , `file.access` and `file_test` with various versions of this test : `file_test` is a version of the POSIX `test` command for those familiar with shell scripting.* » (§14.1)

La parenthèse est le cœur du problème : « exister » et « être accessible » sont **deux questions différentes**. Un fichier peut être présent sur le disque sans que le compte courant ait le droit de le lire.

- `file.exists` — existe-t-il ?
- `file.access` — puis-je y accéder, et comment ?
- `file_test` — la syntaxe du `test` du shell, pour ceux qui la connaissent.

« *For directory listings use `list.files` (also available as `dir`) or `list.dirs` . These can select files using a regular expression : to select by wildcards use `Sys.glob` .* » (§14.1)

Fonction	Langage de sélection
<code>list.files(pattern=)</code>	une expression régulière
<code>Sys.glob()</code>	des jokers de shell

Deux habitudes différentes : `"\\.csv$"` d'un côté, `"*.csv"` de l'autre.

Le piège classique dans le premier cas : **échapper le point**. `"\\.csv$"` apparie l'extension ; `".csv$"` apparierait aussi `xcsv` — le point non échappé signifiant « n'importe quel caractère ».

« **POSIX filesystems are case-sensitive, so `foo.png` and `Foo.PNG` are different files. However, the defaults on Windows and macOS are to be case-insensitive, and FAT filesystems (commonly used on removable storage) are not normally case-sensitive (and all filepaths may be mapped to lower case).** » (§14.2)

Il est coûteux parce qu'il ne se manifeste qu'au déploiement. Un script développé sous Windows ou macOS, où `Donnees.CSV` et `donnees.csv` désignent le même fichier, **échoue sous Linux** — et le message d'erreur dit simplement que le fichier n'existe pas.

La règle : ne jamais compter sur l'insensibilité, et écrire les noms **exactement** comme ils sont sur le disque.

2. **Le séparateur** — « **Almost all the Windows' OS services support the use of slash or backslash ... and R converts the known exceptions to the form required by Windows.** »
3. **Le slash final** — « **The behaviour ... is OS-dependent. Such paths are not valid on Windows ... POSIX-2008 requires such paths to match only directories, but earlier versions allowed them to also match files. So they are best avoided.** »
4. **Les slashes multiples** — `/abc//def` est valide en POSIX et « *treated as if there was only one slash* », mais « **leading double slashes may have a different meaning** ».
5. **UNC** — « **Windows' UNC filepaths ... are not supported, but they may work in some R functions. POSIX filesystems are allowed to treat a leading double slash specially.** »
6. **Relatif à un lecteur** — `d:foo/bar` ; « **It is intended that these work, but the use of absolute paths is safer.** »

« **Functions `basename` and `dirname` select parts of a file path : the recommended way to assemble a file path from components is `file.path`.** » (§14.2)

```
file.path("data", "brut", "mesures.csv")
```

`paste0(dir, "/", nom)` **code en dur un séparateur.** Cela fonctionne souvent — « **almost all the Windows' OS services support the use of slash or backslash** » — mais pas partout

: le cours précise que **R convertit lui-même les exceptions connues**, ce qu'un `paste0` ne fait pas.

`file.path` **fait le bon choix pour le système courant**, et c'est la fonction que le cours qualifie de « *recommended way* » — la seule formulation aussi nette du chapitre.

`normalizePath` : « *On filesystems with links, a single file can be referred to by many filepaths. `normalizePath` will find a canonical filepath.* » (§14.2) — un fichier, **un** nom de référence.

`shortPathName` : « *Windows has the concepts of short ("8.3") and long file names : `normalizePath` will return an absolute path using long file names and `shortPathName` will return a version using short names. The latter does not contain spaces and uses backslash as the separator, so is sometimes useful for exporting names from R.* »

La raison d'être de `shortPathName` est dans la mise en garde : un nom court **ne contient pas d'espace**, ce qui le rend transmissible à un programme extérieur sans protection supplémentaire.

Et `path.expand` fait « *tilde expansion* » — la substitution des répertoires personnels.

« *Functions `system` and `system2` are used to invoke a system command and optionally collect its output. `system2` is a little more general but its main advantage is that it is easier to write cross-platform code using it.* » (§14.3)

« *`system` behaves differently on Windows from other OSes (because the API C call of that name does). Elsewhere it invokes a shell to run the command : the Windows port of R has a function `shell` to do that.* »

Le point décisif : hors Windows, `system` passe par **un shell** — donc jokers, redirections et tubes fonctionnent. **Sous Windows, non.** Un script qui s'appuie sur ce comportement n'est pas portable.

`system2` **sépare l'exécutable de ses arguments** et évite le problème. C'est celui qu'il faut utiliser par défaut.

Deux, données par le §14.3.

1. « *To find out if the OS includes a command, use `Sys.which` , which attempts to do this in a cross-platform way (unfortunately it is not a standard OS service).* » — la commande peut ne pas exister.
2. « *Function `shQuote` will quote filepaths as needed for commands in the current OS.* » — **protéger les arguments**, en particulier les chemins contenant des espaces.

`shQuote` est à la ligne de commande ce que `file.path` est au chemin : une fonction qui applique **les règles du système courant** à votre place.

Et sous Windows, `shortPathName` offre une seconde parade — un nom **sans espace** par construction.

Sans rien faire de particulier.

« *Recent versions of R have extensive facilities to read and write compressed files, often transparently. Reading of files in R is to a very large extent done by connections, and the `file` function ... is able to identify the compression used from the "magic" header of the file.* » (§14.4)

La connexion lit l'en-tête et décide. `read.csv("f.csv.gz")` fonctionne exactement comme `read.csv("f.csv")`.

Les formats reconnus : gzip — « *the type of compression which has been supported for longest ... remains a good general compromise* » ; **compress** — « *becoming rare* » ; **bzip2** et **xz** — « *higher rates of compression (depending on the file, much higher) at the expense of slower decompression and much slower compression* ».

« *some confusion between `xz` and `Lzma` ... R can read files compressed by most versions of either* ».

« *File archives are single files which contain a collection of files, the most common ones being "tarballs" and zip files as used to distribute R packages. R can list and unpack*

both (see functions `untar` and `unzip`) and create both (for zip with the help of an external program). » (§14.4)

Opération	tar	zip
Lister	<code>untar</code>	<code>unzip</code>
Dépaqueter		
Créer		avec un programme externe

L'asymétrie est dans la parenthèse finale. Créer un zip suppose qu'un utilitaire soit installé — d'où l'intérêt de `Sys.which("zip")` avant de tenter l'opération, et d'un message clair si la commande manque.

Flashcards

Question	Réponse
Pourquoi R accède-t-il au système ?	Pour servir de langage de script
Qui s'en sert le plus ?	R lui-même — par exemple pour installer des paquets
Quel objectif de conception ?	Être aussi indépendant de la plateforme que faisable
Créer un fichier, un dossier ?	<code>file.create</code> , <code>dir.create</code>
Leurs équivalents POSIX ?	<code>touch</code> et <code>mkdir</code>
Un fichier temporaire ?	<code>tempfile</code>
Supprimer un fichier ?	<code>file.remove</code>
Supprimer une arborescence ?	<code>unlink</code>
Lister un répertoire ?	<code>list.files</code> , alias <code>dir</code>
Lister des sous-dossiers ?	<code>list.dirs</code>
Selon quel langage sélectionnent-ils ?	Une expression régulière
Et pour des jokers ?	<code>Sys.glob</code>
Savoir si c'est un dossier ?	<code>file.info</code>
Les trois tests d'existence ?	<code>file.exists</code> · <code>file.access</code> · <code>file_test</code>
Que rappelle le cours à ce sujet ?	Un fichier peut exister sans être visible
Lequel imite <code>test</code> du shell ?	<code>file_test</code>
Copier un fichier ?	<code>file.copy</code> — l'analogue de <code>cp</code>
Choisir un fichier interactivement ?	<code>file.choose</code>
Les variantes Windows ?	<code>choose.files</code> , <code>choose.dir</code>
Et dans <code>tcltk</code> ?	<code>tk_choose.files</code> , <code>tk_choose.dir</code>
Afficher, éditer un fichier ?	<code>file.show</code> , <code>file.edit</code>

Question	Réponse
Deux fonctions de liens ?	<code>file.link</code> , <code>sys.readlink</code>
Les systèmes POSIX sont-ils sensibles à la casse ?	Oui
Et Windows, macOS ?	Insensibles par défaut
Et FAT ?	Insensible, parfois tout en minuscules
Windows accepte quel séparateur ?	Les deux — R convertit les exceptions
Un chemin terminé par un slash ?	Non valide sous Windows — à éviter
<code>/abc//def</code> en POSIX ?	Valide , traité comme un seul slash
Un double slash initial ?	Peut avoir un sens différent
Les chemins UNC ?	Non supportés
<code>d:foo/bar</code> ?	Relatif au répertoire courant du lecteur
Que recommande le cours ?	Les chemins absolus
Extraire le nom d'un fichier ?	<code>basename</code>
Extraire son dossier ?	<code>dirname</code>
Assembler un chemin ?	<code>file.path</code> — « <i>the recommended way</i> »
Résoudre un <code>~</code> ?	<code>path.expand</code>
Obtenir un chemin canonique ?	<code>normalizePath</code>
Les noms courts de Windows ?	<code>shortPathName</code> — format 8.3
Leur avantage ?	Pas d'espace
Changer les permissions ?	<code>sys.chmod</code>
Que reconnaît Windows en permissions ?	Seulement le lecture seule
R gère-t-il les ACL ?	Non
Les deux fonctions de commande système ?	<code>system</code> et <code>system2</code>

Question	Réponse
Laquelle pour du code portable ?	<code>system2</code>
Que fait <code>system</code> hors Windows ?	Il invoque un shell
Et sous Windows ?	Il faut <code>shell</code>
Vérifier qu'une commande existe ?	<code>Sys.which</code>
Est-ce un service standard du système ?	Non
Protéger un chemin pour le shell ?	<code>shQuote</code>
Comment R lit-il un fichier compressé ?	Par une connexion
Qui identifie la compression ?	La fonction <code>file</code> , par l' en-tête magique
Le format le mieux équilibré ?	gzip
Le format ancien et rare ?	compress
Les deux formats à forte compression ?	bzip2 et xz
Leur inconvénient ?	Compression bien plus lente
<code>xz</code> et <code>lzma</code> ?	R lit la plupart des versions des deux
Les deux archives les plus courantes ?	Les tarballs et les zip
Comment les dépaqueter ?	<code>untar</code> , <code>unzip</code>
R peut-il créer un zip seul ?	Non — programme externe requis